

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Содержание

Переменные	2
Множественное присваивание	3
Удаление переменной	7
Операторы в Python.....	9
Комбинированные операторы присваивания	9
Операторы сравнения	12
Таблица истинности для логических операторов	14
Операторы тождественности	16
Операторы принадлежности (членства)	17
Общие рекомендации по написанию кода (I).....	18

Переменные

Сразу хочется отметить, что язык программирования Python, как и многие другие языки постоянно развивается и дополняется. Такой эволюционный процесс связан с трудом множества специалистов, они-то и создали стандарты, которым написанный код должен соответствовать. Главная задача таких стандартов - создание общепонятного стиля программирования.

Эти стандарты называются PEP - Python Enhancement Proposals и изучить их можно на [соответствующей странице](#) документации языка python.

PEP 0 – Index of Python Enhancement Proposals

Author: python-dev <python-dev at python.org>

Status: Active

Type: Informational

Created: 13-Jul-2000

Главным стандартом, определяющим стиль написания кода на Python, является [PEP8](#). По [этой](#) ссылке вы найдёте неплохой перевод на русский язык: **Внимательно изучите этот стандарт перед выполнением заданий!**

Объявление переменных. Присваивание значения переменной

В языке Python логика оператора = (оператора присваивания) отличается от привычного нам арифметического символа равенства. Выражения, содержащие этот оператор, нужно интерпретировать следующим образом: *мы присваиваем результат операции, которая выполняется справа от символа равенства, имени, которое находится слева, то есть читаем выражение справа налево.*

Когда мы записываем конструкцию вида

```
age = 10
```

компилятор Python понимает, что число 10 нужно связать с некоей переменной [age](#). Но как только произойдёт какое-то действие с этой переменной, например, мы прибавим к ней 5:

```
age = age + 5
```

первоначальная связь с числом 10 пропадет, и компилятор свяжет [age](#) уже с новым значением 15 (10 + 5), а сущность под названием «сборщик мусора» тут же уничтожит те объекты, на которые больше нет ни одной ссылки в программе, чтобы в памяти не оставалось не используемых объектов.

Обратите внимание – необходимо приучать себя сразу давать переменным осмысленные имена на английском языке, используя змеиную нотацию (*snake_case*), то есть слова в имени переменной пишутся с маленькой буквы, а разделяются нижними подчёркиваниями.

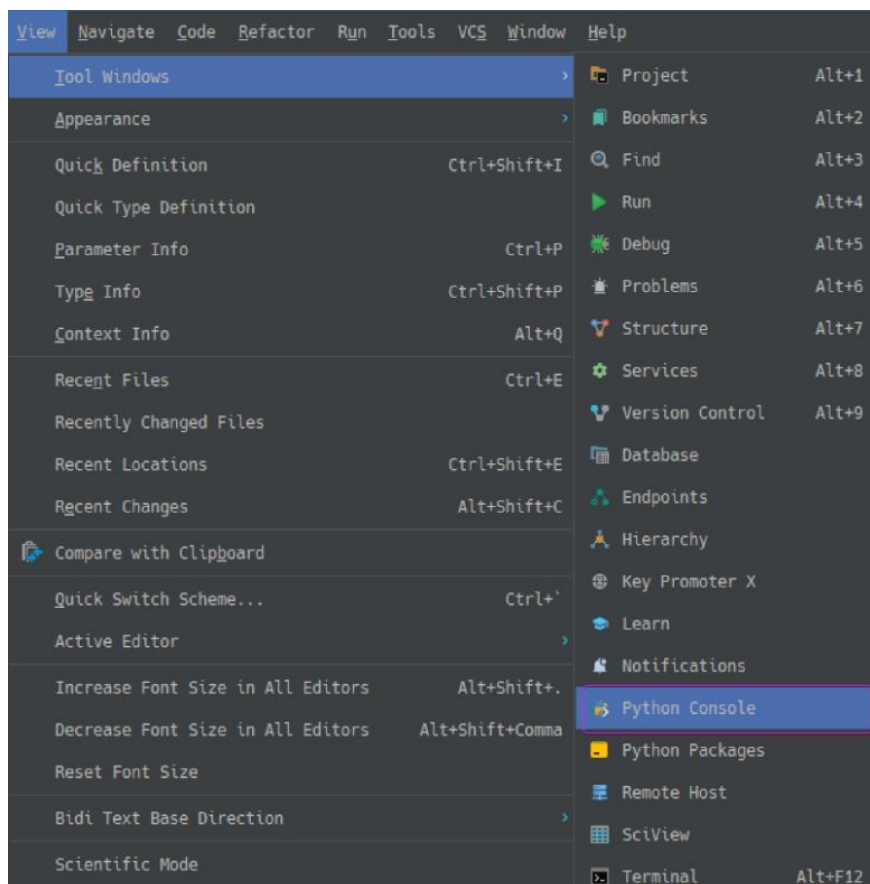
Если необходимо создать переменную содержащую длину какого-то объекта, то ей лучше дать имя `length` или `object_len`, то есть имя должно отражать хотя бы какую-то информацию о том, что находится внутри этой переменной.

Избегайте крайностей, создавая имя длиной 5-6 слов. Старайтесь обходиться без предлогов и артиклей.

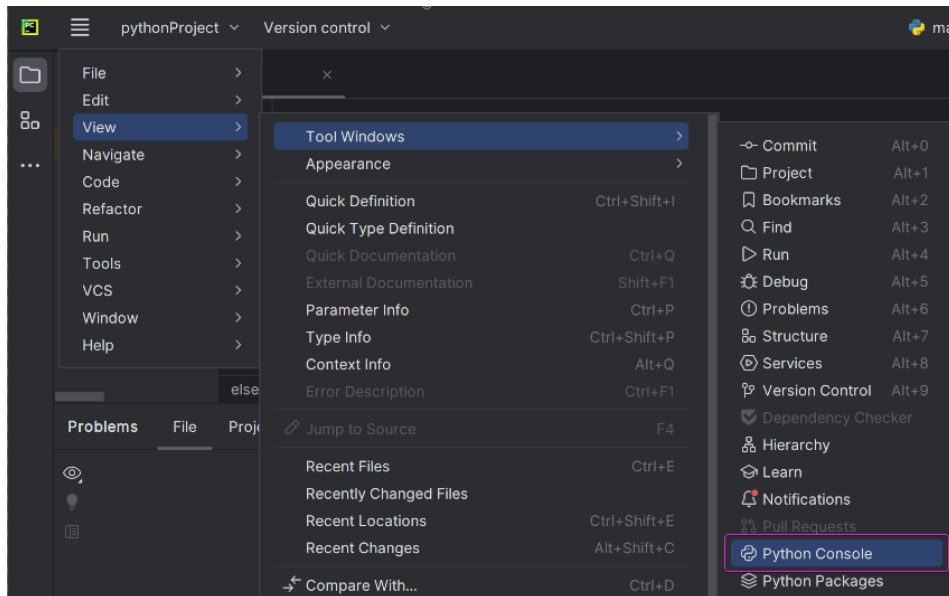
Однобуквенные имена `x` или `y` крайне не рекомендуются для использования. Не лучшим вариантом будет и транслитерация русского слова `длина`. Не знаете, как пишется то или иное слово, открываете любой словарь или translate.google.com, ищете там перевод слова, и используете его.

Выполнение код на языке python

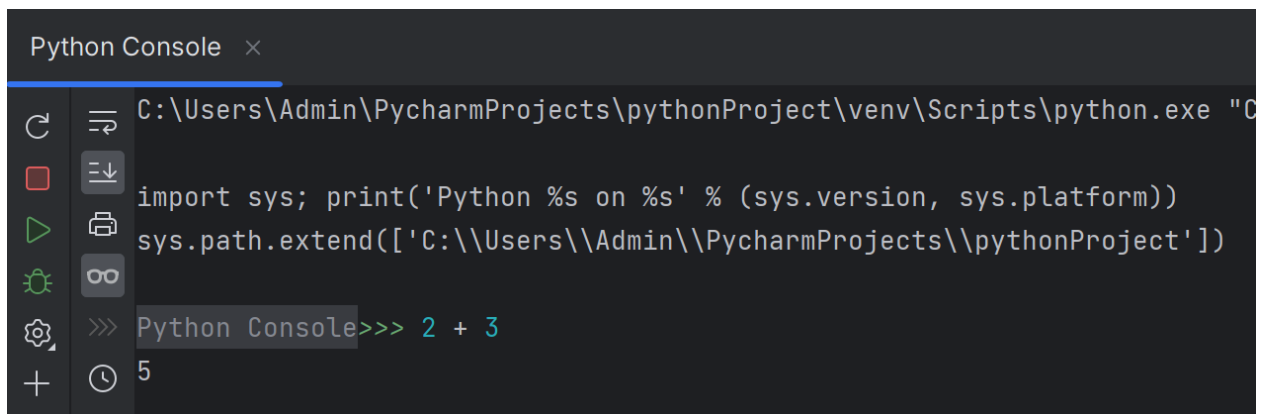
Далее в методичке будет приведён код, обрабатываемый инструментом PyCharm под названием Python Console. Чтобы вызвать его, он находится в меню View – Tool Windows – Python Console



В новых версиях PyCharm путь к нему выглядит немного иначе

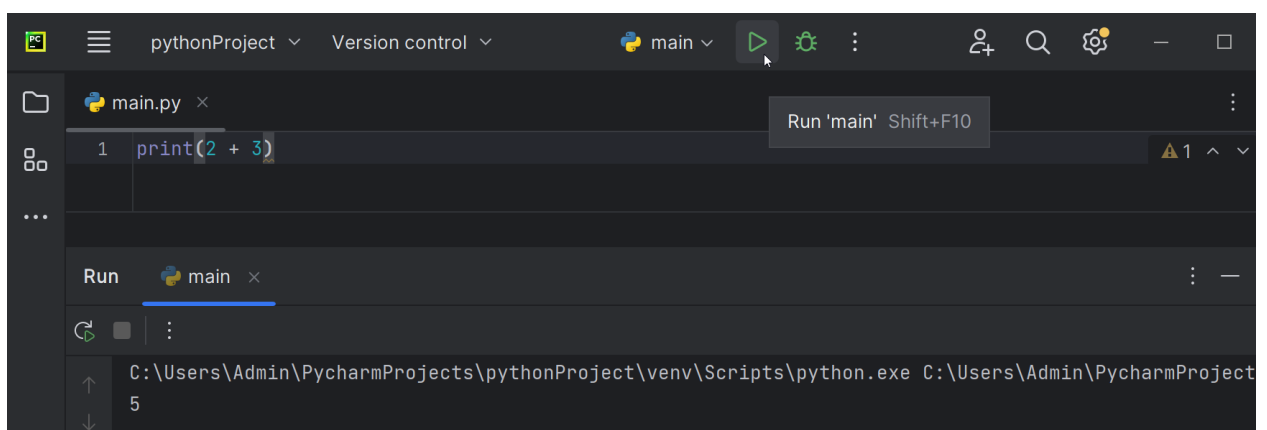


В этой среде python выполняет строку кода после каждого нажатия клавиши Enter.



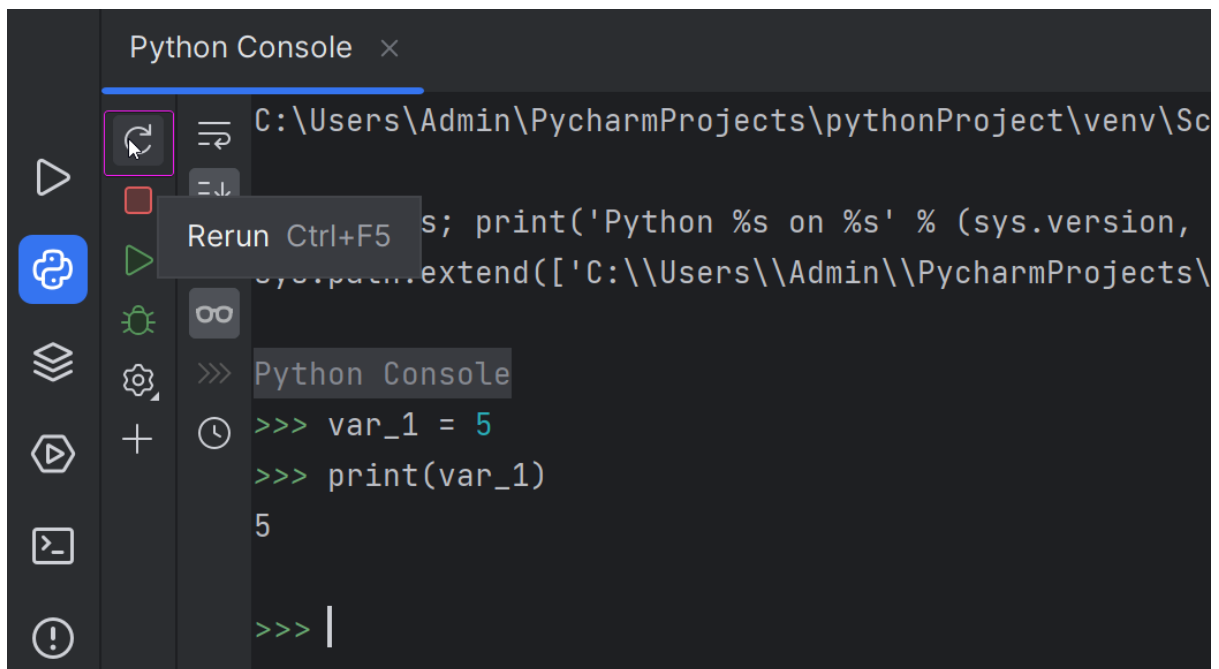
Подобный подход позволяет сразу получить результат операции.

Если же мы работаем в основном окне программы, то выполнение кода будет происходить только в тот момент, когда мы нажимаем на зелёную стрелку или комбинацию клавиш **Shift-F10**



Вы можете одновременно использовать эти два этих режима для написания кода. В основном окне программы писать полноценный код, а в Python Console тестировать выполнение каких-то его блоков. Но обратите внимание – эти два окна никак между собой не связаны. То есть, при объявлении переменной в одном инструменте, она не оказывается в другом – это важно понимать.

И ещё один важный момент – в основном окне программы объекты создаются заново каждый раз при запуске. А вот в Python Console они сохраняются на время работы сессии. Даже, если переменная не видна в окне, она продолжает существовать. Чтобы перезапустить Python Console, начать «с чистого листа», используйте комбинацию клавиш **Ctrl-F5** или кнопку **return** слева



Множественное присваивание

Python позволяет создавать несколько переменных сразу в одной строке кода. Но нужно иметь в виду, что количество объектов справа от оператора присваивания должно быть меньше или равно именам переменным.

```
>>> user, admin, number = 'Anna', 'Bob', 9
>>> print(user, admin, number)
Anna Bob 9
```

Если значений больше, и в одну переменную нужно включить сразу несколько значений, то перед её именем нужно поставить знак звёздочки.

```
>>> *users, age = 'Anna', 'Kate', 'Bob', 9
>>> print(users)
['Anna', 'Kate', 'Bob']
>>> print(age)
9
```

В этом случае переменная `users` будет иметь тип список, о них поговорим в следующих уроках.

Также Python позволяет одновременно назначать одно значение нескольким переменным.

```
>>> var_1 = var_2 = var_3 = 'python'
>>> var_1
'python'
>>> var_2
'python'
```

Одной из особенностей присваивания в python является возможность поменять значения между переменными одной строкой кода, без создания третьей переменной:

```
>>> var_digit, var_string = 10, 'my_string'
... print(var_digit, var_string)
...
... var_digit, var_string = var_string, var_digit
... print(var_digit, var_string)
...
10 my_string
my_string 10
```

В первой строке кода мы создали 2 переменные `var_digit` и `var_string`, присвоив им число десять и строку “my_string” соответственно. Далее распечатали эти переменные и получили результат 10 my_string

Чтобы поменять местами значения между переменными в других языках программирования нужно вводить ещё одну переменную. В python же мы можем использовать конструкцию `var_digit, var_string = var_string, var_digit`, чтобы обменять значения между переменными.

Удаление переменной

В случае, когда мы самостоятельно хотим удалить переменную из памяти программы, мы можем использовать оператор `del`.



Обратите внимание – IDE PyCharm сама подскажет вам о том, что в коде есть какая-то неоднозначная ситуация, которая может привести к ошибке, подсветив переменную жёлтым цветом. Получить более подробную информацию о проблеме можно, наведя курсор на выделенную жёлтым область или, открыв вкладку Problems в нижней части окна


```

1 tmp_var = 'python'
2 print(tmp_var)
3
4 del tmp_var
5 print(tmp_var)
6

```

Name 'tmp_var' can be undefined

main

tmp_var: str = 'python'

PyCharm сообщает, что переменная может быть не определена

```

1 tmp_var = 'python'
2 print(tmp_var)
3
4 del tmp_var
5 print(tmp_var)
6

```

Version Control Run TODO Problems Terminal

1 Расположение кнопки Problems

```

4 del tmp_var
5 print(tmp_var)
6

```

Problems: File 1 Project Errors

main.py /home/explorer/PycharmProjects

Name 'tmp_var' can be undefined :5

2 Описание проблемы

При попытке выполнить этот код, мы сначала увидим распечатанную строку (1), а ниже сообщение об ошибке (2)

```

Run: main x
/home/explorer/PycharmProjects/lessons/venv/bin/python /home/explorer/PycharmProj
python
Traceback (most recent call last):
  File "/home/explorer/PycharmProjects/lessons/main.py", line 5, in <module>
    print(tmp_var)
    ^^^^^^^
NameError: name 'tmp_var' is not defined

```

python сообщает нам, что на строке 5 произошла ошибка – мы попытались обратиться к переменной, которая была удалена. Программа была остановлена, и вывелся Traceback – трассировка вызовов – python показывает всю иерархию обращения к переменной, чтобы мы смогли быстрее понять, в каком же именно месте произошла ошибка.

Практика удаления переменных встречается крайне редко, но знать об этом функционале необходимо.

Операторы в Python

Комбинированные операторы присваивания

Как мы уже знаем, символ `=` является оператором присваивания. Операторы присваивания также бывают комбинированными (составными), они делают код компактнее.

Оператор	Присвоение переменной значения	Сокращённый вариант
<code>+=</code>	<code>a = 2</code> <code>a = a + 5</code>	<code>a = 2</code> <code>a += 5</code>
<code>-=</code>	<code>b = 20</code> <code>b = b - 5</code>	<code>b = 20</code> <code>b -= 5</code>
<code>*=</code>	<code>c = 2</code> <code>c = c * 8</code>	<code>c = 2</code> <code>c *= 8</code>
<code>/=</code>	<code>d = 12</code> <code>d = d / 4</code>	<code>d = 12</code> <code>d /= 4</code>
<code>%=</code>	<code>x = 34</code> <code>x = x % 6</code>	<code>x = 34</code> <code>x %= 6</code>
<code>**=</code>	<code>y = 3</code> <code>y = y ** 3</code>	<code>y = 3</code> <code>y **= 3</code>
<code>//=</code>	<code>z = 22</code> <code>z = z // 4</code>	<code>z = 22</code> <code>z //= 4</code>

```
>>> var_1 = 2
>>> var_2 = 10
>>> var_1 += 3
>>> print(var_1)
5
>>> var_2 *= var_1
>>> print(var_2)
50
```

Старайтесь использовать именно такую – краткую запись, вместо полной.

Арифметические операторы

Арифметические операторы аналогичны тем, что используются на калькуляторе, исключением являются операторы деления: `%` - возвращает остаток от деления одного числа на другое, в то время как целочисленное деление `//` вернёт только целую часть. Если же мы будем использовать для деления стандартный символ `/`, возвращаемое значение всегда будет дробью, даже если остатка нет.

Название оператора	Вид	Использование	Результат
Сложение	<code>+</code>	<code>2 + 2</code>	4
Вычитание	<code>-</code>	<code>5 - 3</code>	2
Умножение	<code>*</code>	<code>2*10</code>	20
Деление	<code>/</code>	<code>100/2</code>	50.0
Целочисленное деление	<code>//</code>	<code>26//5</code>	5
Получение остатка от деления	<code>%</code>	<code>65%20</code>	5
Возведение в степень	<code>**</code>	<code>3**3</code>	27

Не забываем, что математические вычисления имеют приоритеты выполнения.

Математические операции имеют такой приоритет:

1. Выражение в скобках
2. Экспоненты
3. Умножение
4. Деление
5. Сложение
6. Вычитание



Обратите внимание – добавлять пробелы после математических операторов нужно в соответствии с их приоритетами - приоритет умножения, деления и возведения в степень выше - значит, там пробел не нужен. А вот сложение и вычитание - операции с низким приоритетом - там пробел нужен.

Так, например, мы можем получить совершенно разный результат, изменяя приоритеты, добавив скобки в выражение.

```
>>> print(26 - 17*4)
-42
>>> print((26 - 17)*4)
36
```

Необходимо отметить, что складывать мы можем не только цифры, но и строки, и другие типы данных. Для строк, эта операция называется **конкатенацией**:

```
>>> 'code' + 'by'
'codeby'
```

Python так же может умножить строку на какое-то число. В этом случае, мы просто получим повторение этой строки заданное количество раз.

```
>>> '*'*12
'*****'
```



Обратите внимание – в старых версиях Python умножать можно было только строку на число, в ином случае компилятор выводил ошибку. В версиях $\geq 3.7.0$ умножать можно обоими способами.

```
>>> '*'*12
'*****'

>>> 12*'*'
'*****'
```

Строки с числом сложить нельзя, так как они относятся к разным типам данных.

```
>>> 'hacker' + 777
Traceback (most recent call last):
  File "/opt/pycharm-community-2023.1.3/plugins/python-ce,
    coro = func()
          ^^^^^^^
  File "<input>", line 1, in <module>
TypeError: can only concatenate str (not "int") to str
```

Операторы сравнения

Операторы сравнения аналогичны изученным в школьной программе:

Оператор	Описание	Пример	Результат
==	Проверяет, равны ли оба операнда. Если да, то условие становится истинным – True, иначе ложным - False	2 == 2 2 == 5 True == False	True False False
!=	Проверяет, не равны ли оба операнда. Если нет, то условие становится истинным – True	2 != 2 2 != 5 True != False	False True True
>	Проверяет больше ли значение левого операнда, чем значение правого. Если да, то условие становится истинным.	2 > 1 2 > 10 'A' > 'B'	True False False
<	Проверяет меньше ли значение левого операнда, чем значение правого. Если да, то условие становится истинным.	2 < 1 2 < 10 'A' < 'B'	False True True
>=	Проверяет больше или равно значение левого операнда, чем значение правого. Если да, то условие становится истинным.	2 >= 2 2 >= 5 True >= False	True False True
<=	Проверяет меньше или равно значение левого операнда, чем значение правого. Если да, то условие становится истинным.	2 <= 2 2 <= 5 True <= False	True True False

Отдельно стоит отметить, что **сравнение строк** в python происходит посимвольно. То есть python сравнивает первый символ первой строки с первым символом второй строки, если они равны, сравнение переходит ко второму символу. Процесс будет продолжаться до первого отличающегося символа. «Вес» символов берется по их порядковым номерам в таблице кодировки ASCII (для букв английского алфавита и спецсимволов) и из таблицы кодировок UTF8 для остальных алфавитов.

Подобное сравнение называется [лексикографическим](#). Таблица кодировок находится в самом python, а для получения порядкового номера буквы, используется функция `ord()`, а для получения буквы по номеру – `chr()`.

```
>>> ord('X')
88
>>> chr(88)
'X'
```

Логические (булевы) операторы

Забегаая немного вперёд, стоит отметить, что `python` является языком с *динамической типизацией*. То есть, создав переменную `answer`, мы можем «положить» в неё сначала строку, а дальше в коде изменить значение на другой тип – число, список, кортеж и так далее. С одной стороны такой подход делает код более гибким, с другой – ведёт к некоторому числу ошибок. Существуют иные языки программирования, уже со *строгой типизацией* – C, C++, Golang, Rust и прочие. В их коде менять тип данных переменной нельзя – это приведёт к ошибке.

Узнать, какой тип данных находится в какой-то переменной, можно с помощью функции `type`:

```
>>> my_string = "python"
>>> result = type(my_string)
>>> print(result)
<class 'str'>
```

В коде выше мы присвоили переменной `my_string` строку "python", в следующей строке присвоили переменной `result` результат выполнения функции `type`, которой в свою очередь передали переменную `my_string` в качестве аргумента. И в третьей строке вывели результат на печать. Строка `<class 'str'>` говорит нам о том, что объект в переменной `my_string` является строкой (относится к классу строк).

Наряду со строками и числами, в `python` есть логический тип – `bool` (назван так в честь английского математика [Джорджа Буля](#)). Этот тип данных может принимать всего 2 значения истина – `True`, или ложь – `False`.

```
>>> boolean_var = True
>>> print(type(boolean_var))
<class 'bool'>
```

Практически любой объект в `python` можно привести к такому логическому типу и тут нужно запомнить такой момент:



Значение `False` нам вернут следующие объекты: число 0, пустая строка "", пустой кортеж, список, словарь и множество, объект `None` и само значение `False`.

Мы можем привести к булеву типу почти любой объект с помощью функции `bool`:


```
>>> my_string = "python"
>>> result_1 = bool(my_string)
>>> print(result_1)
True
>>> print(result_1 == True)
True
```

В коде выше, после создания переменной `my_string`, мы создали переменную `result_1`, в которую положили результат выполнения функции `bool`, которая, получив в аргументах переменную, вернула её булево значение – `True`. Если бы мы передали ей пустую строку, то результат был бы иной:

```
>>> result_1 = bool("")
>>> print(result_1)
False
>>> print(result_1 == True)
False
```

Таблица истинности для логических операторов

Оператор	Описание	Пример	Результат
and	Логический оператор "И". Возвращает значение <code>True</code> если оба утверждения верны.	<code>True and True</code>	<code>True</code>
		<code>True and False</code>	<code>False</code>
		<code>False and True</code>	<code>False</code>
		<code>False and False</code>	<code>False</code>
or	Логический оператор "ИЛИ". Возвращает <code>True</code> если одно из утверждений верно.	<code>True or True</code>	<code>True</code>
		<code>True or False</code>	<code>True</code>
		<code>False or True</code>	<code>True</code>
		<code>False or False</code>	<code>False</code>
not	Логический оператор "НЕ". Возвращает <code>False</code> если результат <code>True</code> .	<code>not True</code>	<code>False</code>
		<code>not False</code>	<code>True</code>

```
>>> int_var = 11
>>> bool_var = (int_var < 5 and int_var < 10)
>>> print(bool_var)
False
>>> bool_var = not (int_var < 5 and int_var < 10)
>>> print(bool_var)
True
>>> bool_var = (int_var < 5 or int_var < 10)
>>> print(bool_var)
False
>>> bool_var = (int_var < 5 or int_var > 10)
>>> print(bool_var)
True
```

Операторы тождественности

Операторы тождественности используются для сравнения объектов. Являются ли они одним и тем же объектом, с одинаковым местоположением в памяти.

is	Возвращает True если переменные являются одним объектом.	<code>x is y</code> вернет истину, если <code>id(x)</code> будет равно <code>id(y)</code>
is not	Возвращает True , если переменные от разных объектов.	<code>x is not y</code> , вернет истину если <code>id(x)</code> не равно <code>id(y)</code>

```
>>> int_1 = 11
>>> int_2 = 11
>>> print(int_1 is int_2)
True
>>> print(int_1 is not int_2)
False
```

Функция `id` как раз и позволяет нам узнать номер ячейки в памяти, в которой хранится ссылка на нашу переменную.

```
>>> id(int_1)
94514089549800
>>> id(int_2)
94514089549800
>>> id(int_2) == id(int_2)
True
```



Обратите внимание – для того, чтобы лишний раз не обращаться к памяти, некоторые объекты в python находятся в кэше. Кэш – это определённый раздел памяти, доступ к которому осуществляется гораздо быстрее, нежели ко всей остальной памяти.

Среди прочих объектов там находятся числа от 0 до 127, поэтому их `id` меняться не будет. Сравните 2 вывода:

```
>>> var_1 = var_2 = 500
>>> var_3 = 500
>>> var_1 == var_3
True
>>> id(var_1) == id(var_2)
True
>>> id(var_1) == id(var_3)
False
>>> print(id(var_1), id(var_3))
140322449845328 140322449841424
```

```
>>> var_1 = var_2 = 10
>>> var_3 = 10
>>> var_1 == var_3
True
>>> id(var_1) == id(var_2)
True
>>> print(id(var_1), id(var_2))
94514089549768 94514089549768
```

Значения переменных `var_1` и `var_3` в первом случае будут равны, а вот расположение в памяти – нет. Если же объекты находятся в кэше, то и значения, и `id` будут у них одинаковыми, что подтверждает правый скриншот.

Операторы принадлежности (членства)

in Возвращает истину, если элемент присутствует в последовательности

not in Возвращает истину, если элемента нет в последовательности

```
>>> codeby_str = 'Codeby'
>>> print('Cod' in codeby_str)
True
>>> print('cod' in codeby_str)
False
```

Оператор принадлежности является очень мощным инструментом для проверки различных условий вхождения. Он используется для тех типов данных, которые представляют собой последовательности – в примере выше это – строка. К такому же типу относятся списки, кортежи, словари, множества и производные от этих типов. Об этом мы будем говорить в следующих уроках.

Общие рекомендации по написанию кода (I)

Друзья, в этом уроке вам предстоит выполнить первое полноценное домашнее задание, где будет необходимо написать несколько небольших программ. Для того, чтобы код был более читабельный, понятный и поддерживаемый, старайтесь придерживаться следующих правил, которые я буду проверять вместе с правильностью вычислений и самого кода:

1. Как уже было сказано, **не используйте однобуквенные переменные**. Вместо `x`, `y`, `i`, или `z` вводите понятные имена, соответствующие по смыслу переменной. В тех случаях, когда мы считаем количество, это может быть `target_count`, `answer_count` или `answer_cnt`, `obj_qnt` (от object quantity). Для имени переменной используйте хотя бы пару слов, таким образом вы во много раз повысите читаемость вашего кода и вам же не придётся вспоминать, что находится в этой переменной. Пока наш код не очень объёмный, мы можем поднять взгляд выше и увидеть место объявления переменной, но, как только код превысит пару десятков строк, ориентироваться в нём будет сложнее и каждый раз пролистывать экран для поиска инициализации переменной будет очень неудобно. Так что вводите понятные, читаемые имена переменных на английском языке в змеиной нотации
2. Для каждого объекта из условия задания **необходимо создать свою переменную**, дать ей понятное имя и использовать именно это имя в коде. **Использовать числа, не нужно, мы работаем с переменными**.
3. **Отделяйте пустыми строками логические блоки кода** – это объявления переменных, вычисления, вывод результатов на экран. Сравните 2 примера:

```
... string_var = 'python'
... my_var = True
... another_var = var_1*string_var
... var_1 *= 20
... print(var_1)
... print(my_var, another_var)
```

неправильно

```
>>> var_1 = 10
... string_var = 'python'
... my_var = True
...
... another_var = var_1*string_var
... var_1 *= 20
...
... print(var_1)
... print(my_var, another_var)
```

правильно

В первом примере код «склеен» и воспринимать его сложно, во втором – мы видим чёткое деление логики. Сначала идёт объявление переменных, потом мы выполняем какие-то действия с ними, а потом выводим на экран результаты наших вычислений.

4. Отсюда же вытекает и следующее правило – **соблюдайте порядок в коде**: сначала мы объявляем все переменные, которые потребуются нам в коде, потом производим с ними какие-то операции, например, вычисления и уже после этого выводим результаты на экран. **Перемешивать эти блоки не нужно.**
5. Повторюсь про пробелы рядом с математическими операторами: операторы с низким приоритетом – сложение (+) и вычитание (-) мы оформляем с пробелами, остальные – без них.

```
>>> print(1+2*3)
7
```

неправильно

```
>>> print(1 + 2*3)
7
```

правильно

6. Не стоит убирать все вычисления в функцию print. То есть **сначала создайте свою переменную для получения результата**, а потом уже выводите её на экран с помощью print'a.

```
>>> print('Правильный ответ: ', 1 + 2)
Правильный ответ: 3
```

неправильно

```
>>> side_a = 1
... side_b = 2
...
... answer = side_a + side_b
...
... print('Правильный ответ: ', answer)
Правильный ответ: 3
```

правильно

7. Для многострочного вывода используйте несколько вызовов функции print, вместо одного с переносами:

```
>>> answer_1 = answer_2 = answer_3 = 'ответ'
... print('Первый ответ задания:', answer_1, '\nВторой ответ задания:
...      answer_2, '\nТретий ответ задания:', answer_3)
Первый ответ задания: ответ
Второй ответ задания: ответ
Третий ответ задания: ответ
```

неправильно

```
>>> answer_1 = answer_2 = answer_3 = 'ответ'
... print('Первый ответ задания:', answer_1)
... print('Второй ответ задания:', answer_2)
... print('Третий ответ задания:', answer_3)
Первый ответ задания: ответ
Второй ответ задания: ответ
Третий ответ задания: ответ
```

правильно

8. Старайтесь писать код **как можно более развёрнуто**, то есть при вычислении площади каких-то геометрических фигур, для каждой стороны создавайте свою переменную и уже эти переменные используйте для вычислений.
9. **Добавляйте текст для описания ответа** в функцию print – как показано в скриншоте справа.
10. Не используйте те знания, до которых мы ещё не дошли. Для выполнения всех заданий достаточно только тех материалов, что даны в текущем и прошлых уроках. Не забегайте вперёд.